Sprint 5: MongoDB

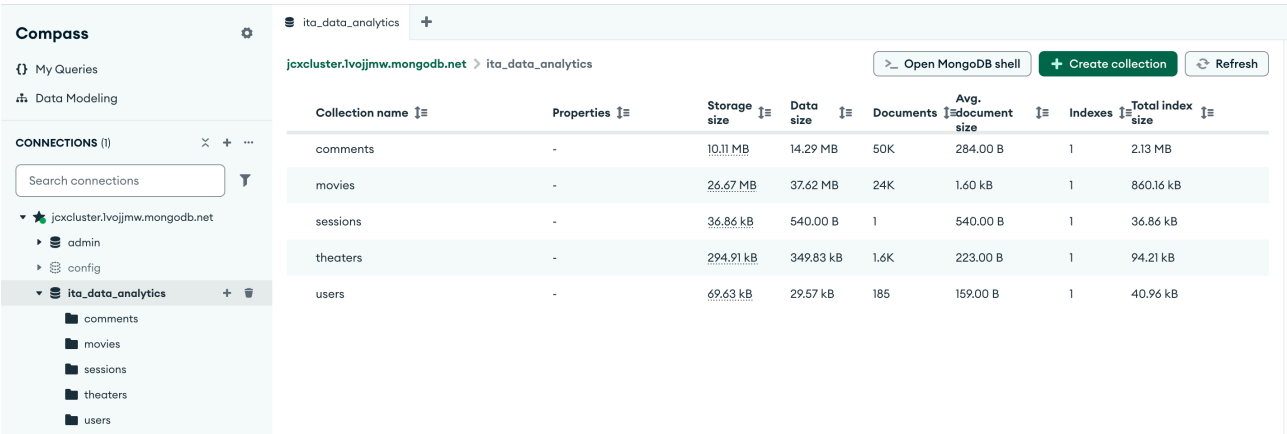
Jordi Calmet Xartó - ITAAD

https://github.com/jordi-cx/it_academy/tree/main/data_analytics/sprint_05

Nivell 1:

Crea una base de dades amb MongoDB utilitzant com a col·leccions els arxius adjunts.

Después de hacer la instalacion de MongoDB, he importado los archivos a través de Compass en la base de datos y en diferentes Colecciones:



Collection name	Properties	Storage size	Data size	Documents	Avg. document size	Indexes	Total index size
comments	-	10.11 MB	14.29 MB	50K	284.00 B	1	2.13 MB
movies	-	26.67 MB	37.62 MB	24K	1.60 kB	1	860.16 kB
sessions	-	36.86 kB	540.00 B	1	540.00 B	1	36.86 kB
theaters	-	294.91 kB	349.83 kB	1.6K	223.00 B	1	94.21 kB
users	-	69.63 kB	29.57 kB	185	159.00 B	1	40.96 kB

Exercici 1.1:

- *Mostra els 2 primers comentaris que hi ha en la base de dades.*

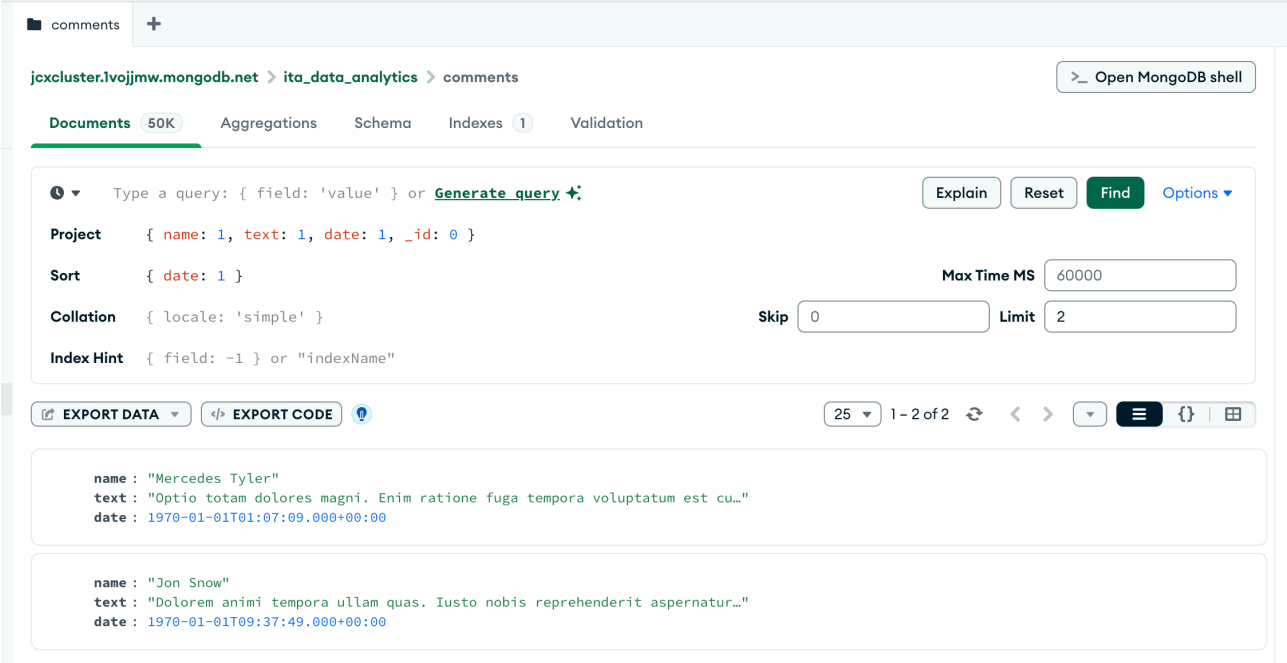
Al abrir la colección con los datos de *comments*, de entrada puedo ver estos dos primeros documentos tal como salen sin ordenar:

```
_id: ObjectId('5a9427648b0beebeb69579cc')
name : "Andrea Le"
email : "andrea_le@fakegmail.com"
movie_id : ObjectId('573a1390f29313caabcd418c')
text : "Rem officiis eaque repellendus amet eos doloribus. Porro dolor volupta…"
date : 2012-03-26T23:20:16.000+00:00
```

```
_id: ObjectId('5a9427648b0beebeb69579cf')
name : "Greg Powell"
email : "greg_powell@fakegmail.com"
movie_id : ObjectId('573a1390f29313caabcd41b1')
text : "Tenetur dolorum molestiae ea. Eligendi praesentium unde quod porro. Co…"
date : 1987-02-10T00:29:36.000+00:00
```

Pero aplicando el siguiente filtro de ordenación ascendente según la fecha (*date*) en la interfaz de Compass, tenemos que estos serían los dos primeros comentarios (cronológicamente) mostrando sólo los campos que nos interesen (por ejemplo, nombre, texto y fecha):

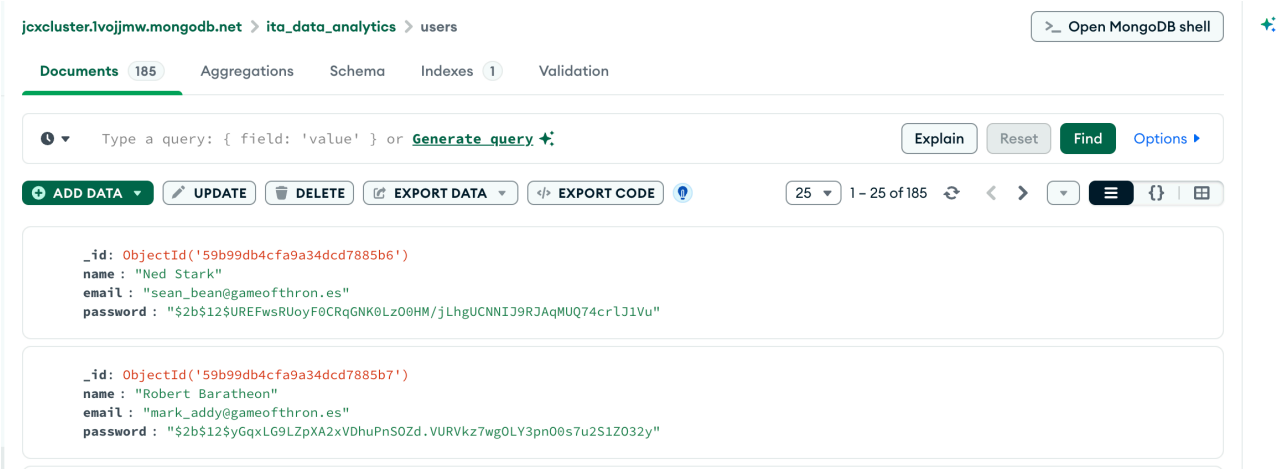
```
// Project:
{ name: 1, text: 1, date: 1, _id: 0 }
// Sort:
{ date: 1 }
// Limit:  2
```




```
{
"name": "Mercedes Tyler",
"text": "Optio totam dolores magni. Enim ratione fuga tempora voluptatum
est cumque animi. Quia nam doloribus corrupti id voluptate esse. Tempore
maiores omnis aliquam ad quisquam quidem.",
"date": "1970-01-01T01:07:09.000Z"
}

{
"name": "Jon Snow",
"text": "Dolorem animi tempora ullam quas. Iusto nobis reprehenderit
aspernatur cupiditate a tempore. Commodi alias expedita dolore explicabo
ipsam.",
"date": 1970-01-01T09:37:49.000Z"
}
```

- *Quants usuaris tenim registrats?*

Mirando en la colección users puedo ver que hay exactamente **185** registros de usuarios:




```
db.users.countDocuments()
```

```
> db.users.countDocuments()
< 185
```

En principio todos estos documentos representan 185 usuarios, pero para comprobar si hay algún campo vacío podríamos por ejemplo ejecutar:

```
{ email: { $ne: null } }
```

O desde la **MongoDB Shell** podemos también ejecutar este comando para ver cuántos emails disitintos hay en la base de datos:

```
db.users.distinct("email").length
```

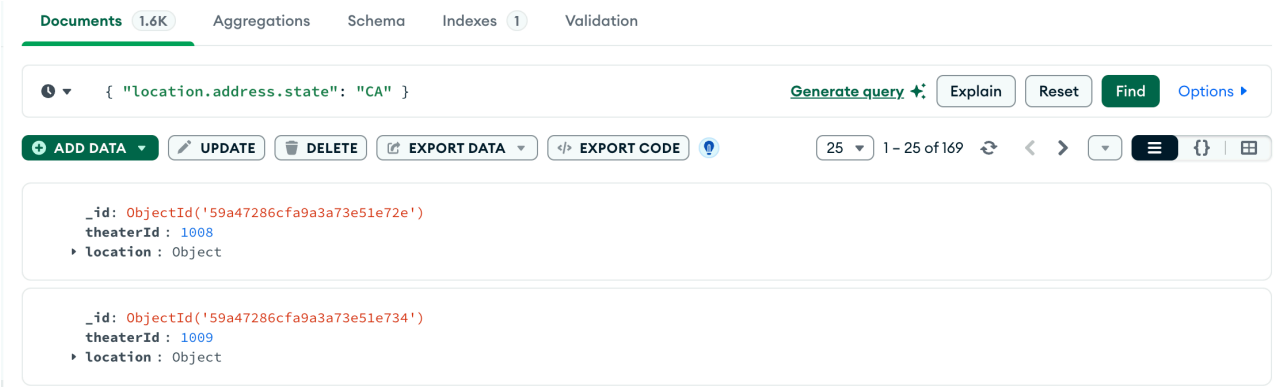
```
> db.users.distinct("email").length
< 185
```

En todos los casos, he comprobado que efectivamente tenemos 185 usuarios.

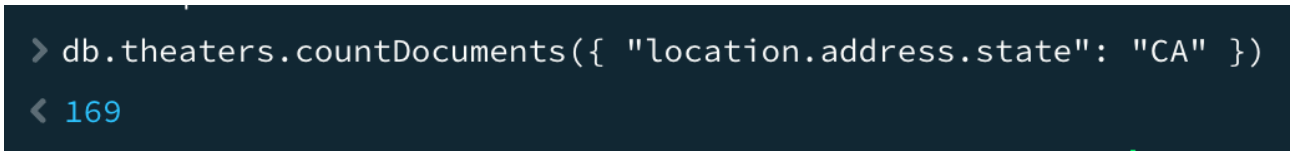
- *Quants cinemes hi ha en l'estat de Califòrnia?*

Aplicando el siguiente filtro en la query, vemos que hay **169 cines** en el estado de California (CA):

```
{ "location.address.state": "CA" }
```



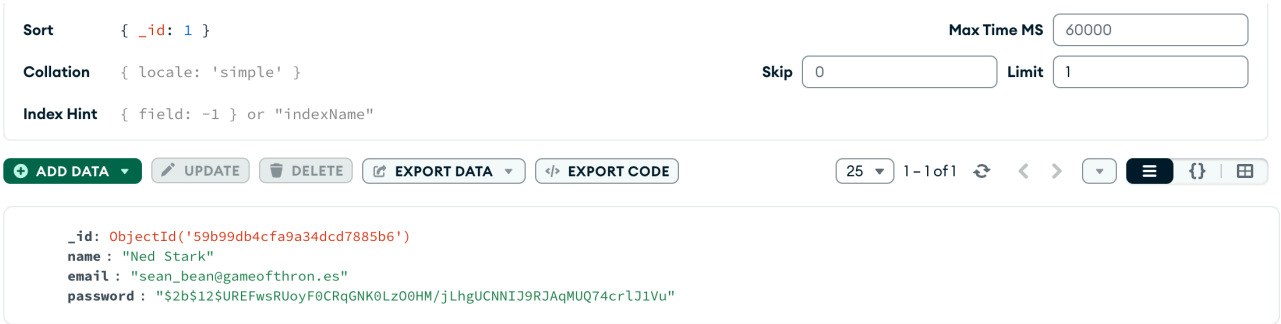

```
// Código MongoDB Shell:

db.theaters.countDocuments({ "location.address.state": "CA" })
```



- *Quin va ser el primer usuari/ària en registrar-se?*

Para determinar el usuario más antiguo, aplico un Sort en sentido ascendente en el campo *_id* (puesto que vemos que se trata de un ID auto-incremental), y aplicamos un límite 1 para quedarnos con el primero:

```
{ _id: 1 }
```



Se trata pues del usuario:

```
"_id": "$oid": "59b99db4cfa9a34dcd7885b6"
"name": "Ned Stark"
"email": "sean_bean@gameofthron.es"
```

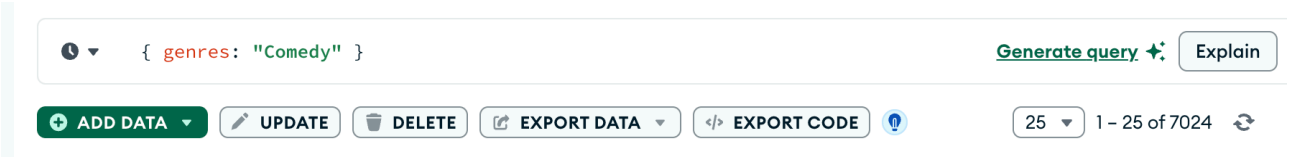
```
// Código MongoDB Shell:

db.users.find({}).sort({ _id: 1 }).limit(1)
```

```
> db.users.find({}).sort({ _id: 1 }).limit(1)
< {
    _id: ObjectId('59b99db4cfa9a34dcd7885b6'),
    name: 'Ned Stark',
    email: 'sean_bean@gameofthron.es',
    password: '$2b$12$UREFwsRUoyF0CRqGNK0LzO0HM/jLhgUCNNIJ9RJAqMUQ74crlJ1Vu'
  }
```

- *Quantes pel·lícules de comèdia hi ha en la nostra base de dades?*

Aplico el siguiente filtro para ver cuántas peliculas incluyen *'Comedy'* en su *Array* de *genres*, y obtengo que son **7024**.

```
{ genres: "Comedy" }
```




```
// Código MongoDB Shell:
db.movies.countDocuments({ genres: "Comedy" })
```

```
> db.movies.countDocuments({ genres: "Comedy" })
< 7024
```

Exercici 1.2:

Mostra'm tots els documents de les pel·lícules produïdes en 1932, però que el gènere sigui drama o estiguin en francès.

He usado la siguiente query para filtrar las películas con las condiciones que piden, y obtengo como resultado **15** documentos:

```
{ year: 1932, $or: [ { genres: "Drama" }, { languages: "French" } ] }
```

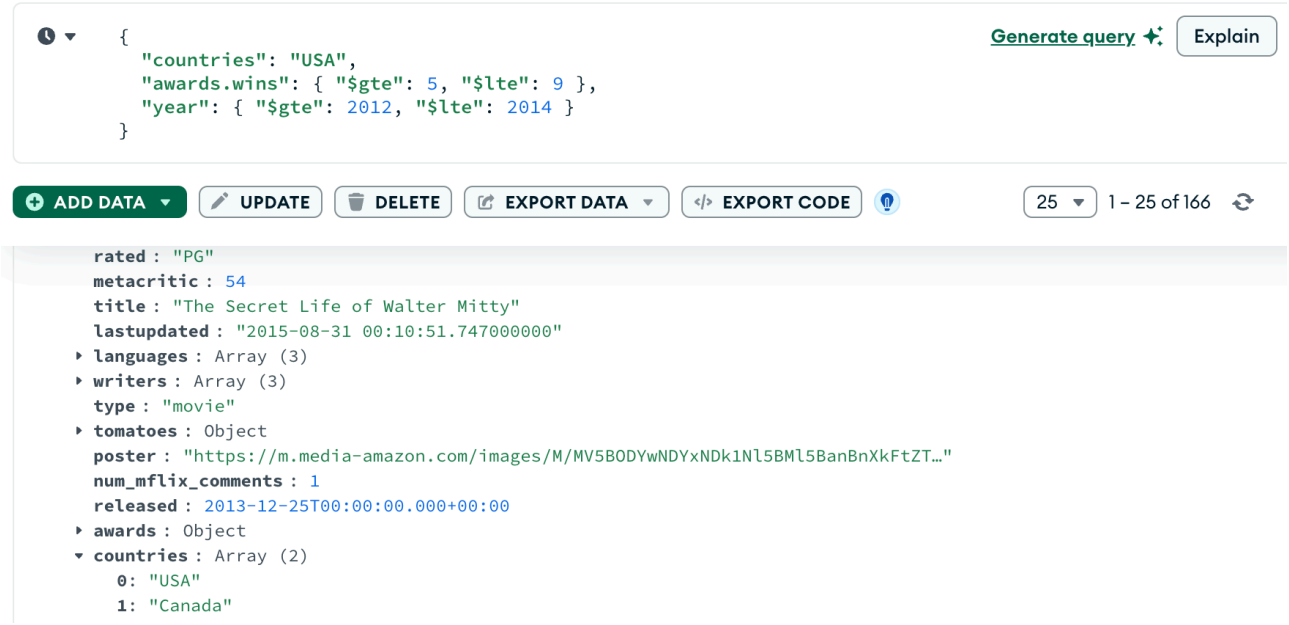
```
// Código MongoDB Shell:

db.movies.find({
  year: 1932,
  $or: [
    { genres: "Drama" },
    { languages: "French" }
  ]
})
```

Exercici 1.3:

Mostra'm tots els documents de pel·lícules estatunidenques que tinguin entre 5 i 9 premis que van ser produïdes entre 2012 i 2014.

Ahora uso el siguiente filtro para ver los rangos de datos que nos piden:

```
{
      "countries": "USA",
      "awards.wins": { "$gte": 5, "$lte": 9 },
      "year": { "$gte": 2012, "$lte": 2014 }
}
```



Obtenemos **166 películas** que cumplen estas condiciones.

```
// Código MongoDB Shell:

db.movies.countDocuments({
  "countries": "USA",
  "awards.wins": { "$gte": 5, "$lte": 9 },
  "year": { "$gte": 2012, "$lte": 2014 }
})
```

```
> db.movies.countDocuments({
    "countries": "USA",
    "awards.wins": { "$gte": 5, "$lte": 9 },
    "year": { "$gte": 2012, "$lte": 2014 }
  })
< 166
```

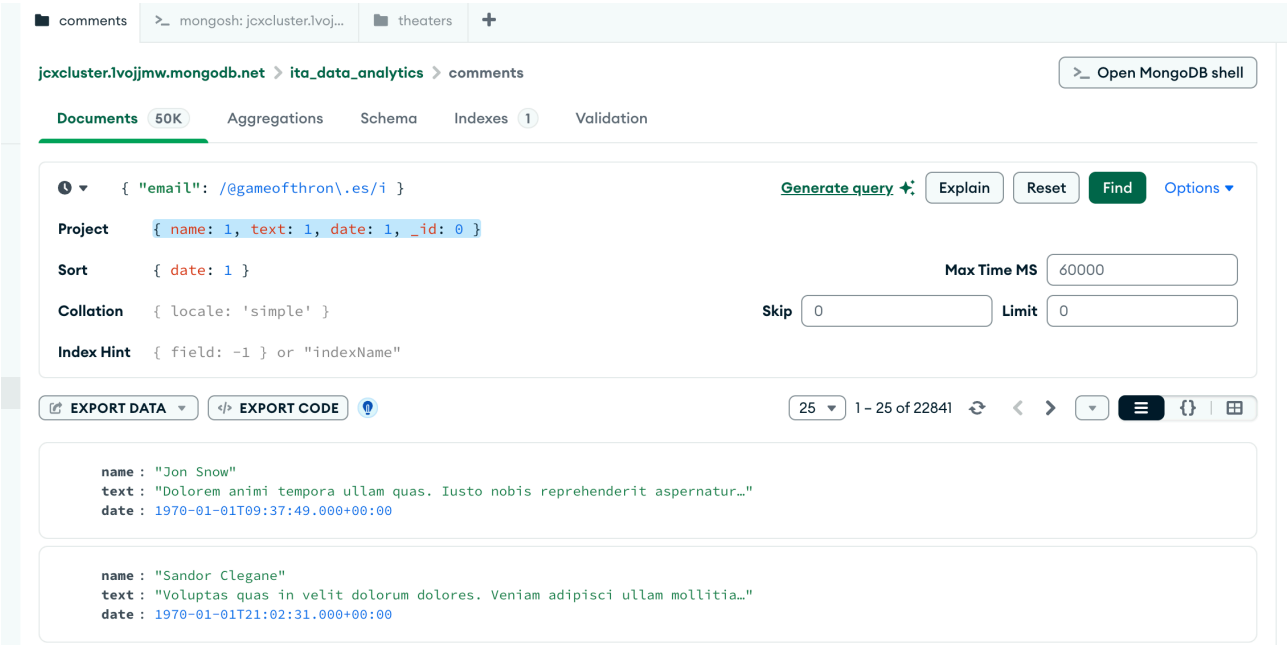
Nivell 2:

Exercici 2.1:

Compta quants comentaris escriu un usuari/ària que utilitza "GAMEOFTHRON.ES" com a domini de correu electrònic.

Aquí miramos en el campo email de los *comments* para ver cuántos incluyen el dominio '@gameoftheron.es' (case insentive), y encontramos **22841 comentarios**:

```
{ "email": /@gameofthron\.es/i }
```




```
// Código MongoDB Shell:

db.comments.countDocuments({ "email": { "$regex": "@gameofthron\\.es",
                                        "$options": "i" } })
```

```
> db.comments.countDocuments({ "email": { "$regex": "@gameofthron\\.es", "$options": "i" } })
< 22841
```

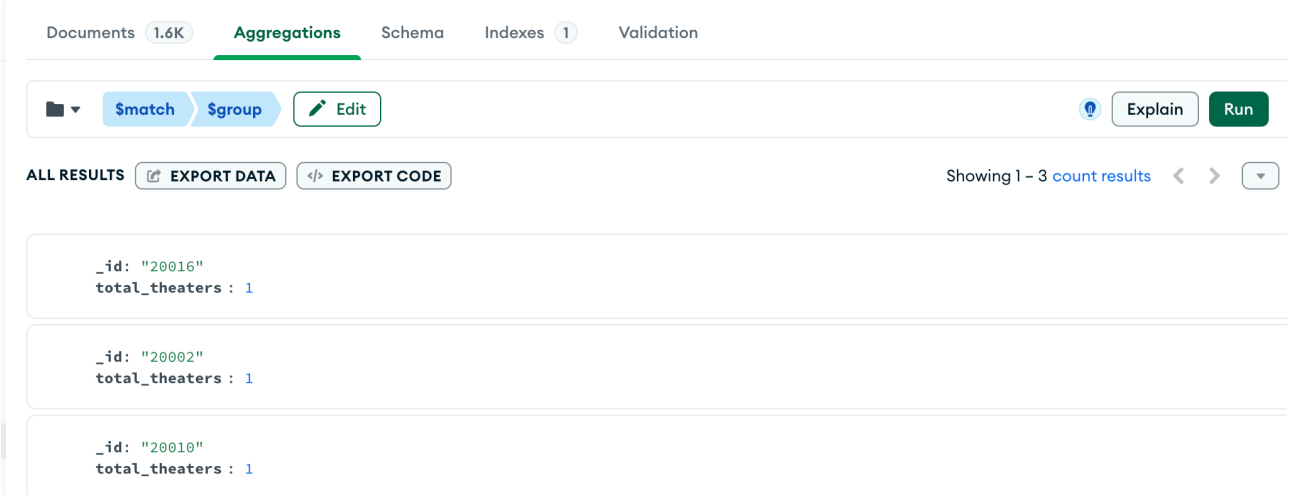
Exercici 2.2:

Quants cinemes hi ha en cada codi postal situats dins de l'estat Washington D. C. (DC)?

Aquí he realizado una **agregación** en dos pasos: primero filtramos por el estado DC y luego agrupamos por el *zipcode* y contamos los resultados.

Me salen estos 3 cines el estado de Washington DC con **3 códigos postales** diferentes:

```
// Aggregation -> Add Stage: $match
{ "location.address.state": "DC" }

// Aggregation -> Add Stage: $group
{
  "_id": "$location.address.zipcode",
  "total_theaters": { "$sum": 1 }
}
```




```
// Código MongoDB Shell:

db.theaters.aggregate([
  // Stage 1: Filter to only include DC
  {
    $match: { "location.address.state": "DC" }
  },
  // Stage 2: Group by zip code and count
  {
    $group: {
      _id: "$location.address.zipcode",
      total_theaters: { $sum: 1 }
    }
  }
])
```

```
{
  _id: '20010',
  total_theaters: 1
}
{
  _id: '20002',
  total_theaters: 1
}
{
  _id: '20016',
  total_theaters: 1
}
```

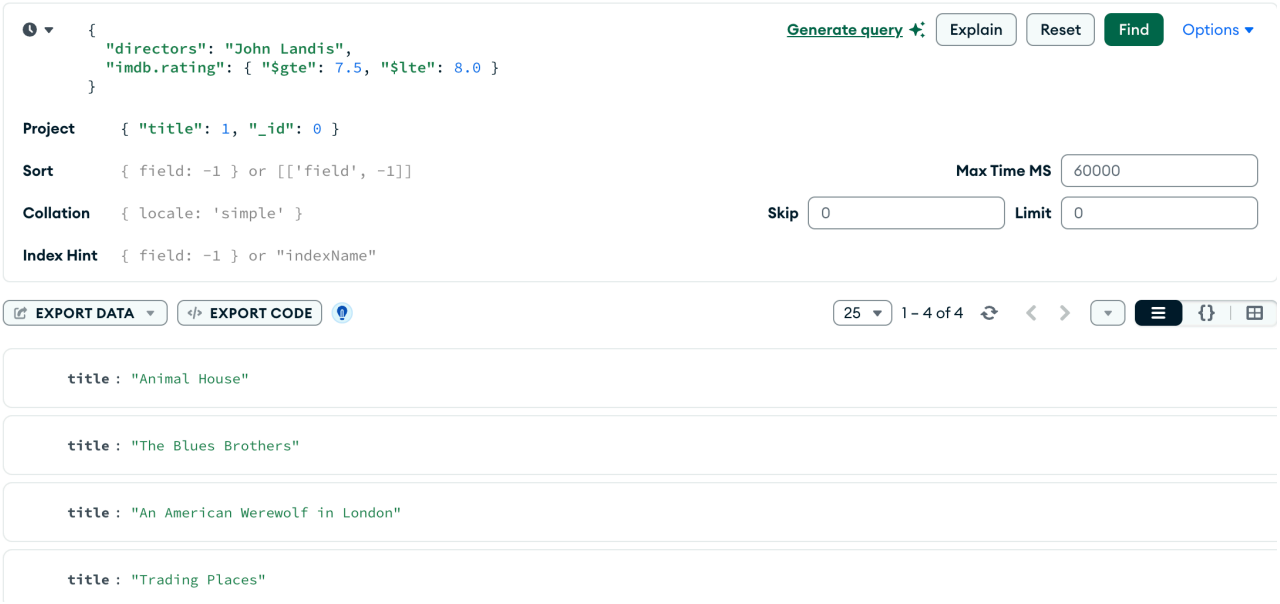
Nivell 3:

Exercici 3.1:

Troba totes les pel·lícules dirigides per John Landis amb una puntuació IMDb (Internet Movie Database) d'entre 7,5 i 8.

Ponemos la siguiente condición en el filtro de la query y luego me quedo con el campo title para ver sólo los títiulos de las películas que buscamos. Son estas **4 peliculas**:

```
// Filter:
{
      "directors": "John Landis",
      "imdb.rating": { "$gte": 7.5, "$lte": 8.0 }
}

// Project:
{ "title": 1, "_id": 0 }
```



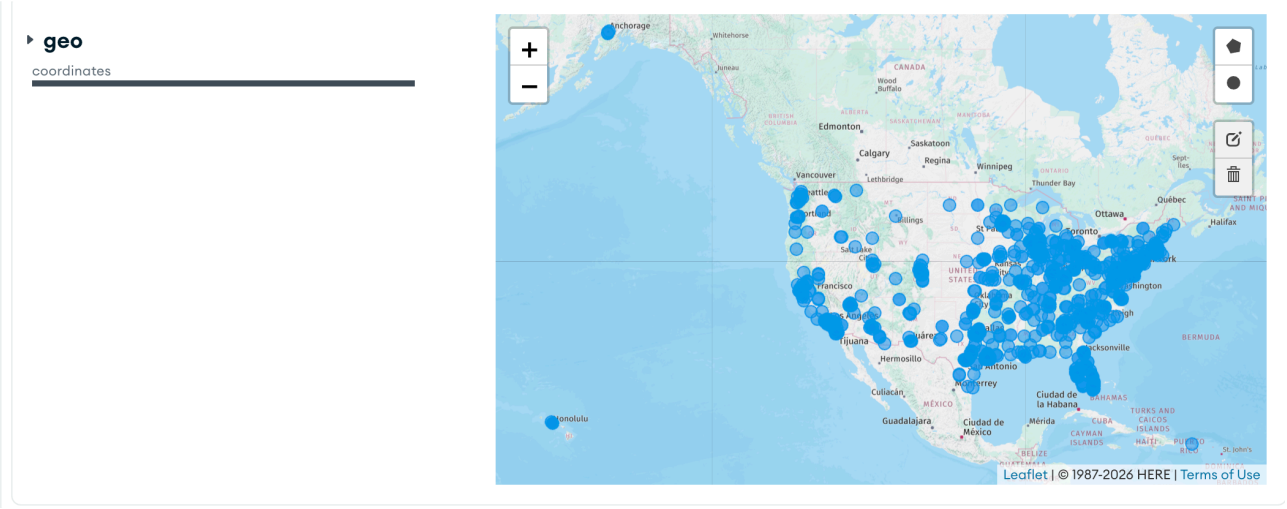

```
// Código MongoDB Shell:

db.movies.find(
  {
    "directors": "John Landis",
    "imdb.rating": { "$gte": 7.5, "$lte": 8.0 }
  },
  {
    "title": 1,
    "_id": 0
  }
)
```

```
{
  title: 'Animal House'
}
{
  title: 'The Blues Brothers'
}
{
  title: 'An American Werewolf in London'
}
{
  title: 'Trading Places'
}
```

Exercici 3.2:

Mostra en un mapa la ubicació de tots els teatres de la base de dades.

En Compass, podemos seleccionar la colección *theaters* y visualizar todos los cines en un mapa mediante ***Schema -> Analyze Schema*** y a traves de las coordenadas incluidas en el campo ***geo***:



REVISIÓN (Gledson):

He hecho la revisión peer-to-peer de mis ejercicios y hemos visto que los resultados estaban correctos.

Lo único importante a comentar es que yo había realizado la mayoría de consultas a través del interfaz Compass y mi peer se había basado casi todo en código de la **MongoDB Shell**. Por eso, incluyo también como serían las instrucciones en Mongosh.